

# Ponytail & Caveman

---

*Do agent “token-saving” skills actually save tokens without quietly moving the cost elsewhere?*

*A repository-level review of implementation, claims, installation, benchmarks, independent evidence, failure modes and a reproducible evaluation protocol.*

## **Research question**

Ponytail and Caveman are both marketed as ways to make AI coding agents leaner. This report asks a harder question: what exactly is being compressed, what does the agent lose, and do savings survive once quality, tool calls, reasoning, input context, cache behaviour and task completion are counted?

Prepared: 12 August 2026

Evidence cut-off: public sources inspected up to 12 August 2026. Repository popularity counts are a point-in-time snapshot and can change quickly.

# Executive verdict

The two projects solve different problems, despite being discussed in the same “token saver” category. Ponytail is primarily a code-minimisation policy: it tries to stop an agent from building unnecessary software. Caveman began as an output-style policy: it makes the agent speak less. Its current repository now also contains a much more ambitious input-compression stack, Caveman Proxy/Engine, which changes the context sent to the model and provides a recovery path. Treating all three mechanisms as one thing produces misleading comparisons.

| Question                  | Ponytail                                                                                                                                     | Caveman skill                                                                                                        | Caveman Proxy / Engine                                                                                                                                       |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Primary mechanism         | Behavioural constraint on implementation choices                                                                                             | Telegraphic natural-language output style                                                                            | Content-aware transformation of input/tool payloads before provider call                                                                                     |
| What it directly reduces  | Often code written; sometimes downstream tokens/time                                                                                         | Assistant prose output                                                                                               | Provider-visible input payloads; tool results, logs, structured content, some skill bodies                                                                   |
| Best independent evidence | 80 paired SkillsBench tasks: ~15% less persisted/emitted code, 10.3% lower cost, 11% faster; no detectable verifier-quality difference [S06] | 82 paired SkillsBench tasks: 8.5% fewer output tokens with forced activation; no detectable quality difference [S13] | No comparably broad independent benchmark located; official pinned six-case benchmark reports 33.2% lower provider input at 18/18 exact-answer quality [S11] |
| Headline vs reality       | 54% code reduction is real on over-build traps but not representative of every task                                                          | 65% output reduction is plausible on prose-heavy prompts but transfers poorly to agentic coding                      | 33.2% is credible for the pinned fixture suite, not a universal saving                                                                                       |
| Main risk                 | Over-minimisation: omitting useful extensibility, checks or maintainability work if scope is misread                                         | Lost nuance / readability; savings small when code, tools and reasoning dominate                                     | Silent context loss, recovery overhead, modality/tokenisation shifts, extra proxy complexity                                                                 |
| My confidence             | Moderate-high that it can reduce bloat; moderate on universal cost benefit                                                                   | High that it makes prose shorter; low that 65% is representative end-to-end                                          | Moderate that structural compression can help on large repetitive payloads; low on universal percentage                                                      |

## Bottom line

Ponytail is the more convincing “engineering discipline” tool: its value does not depend on token arithmetic alone. Caveman’s original skill is useful if you prefer terse agent commentary, but the 65% number should not be used for budgeting. Caveman Proxy is technically more interesting and potentially more valuable, but it also introduces the exact trade-off you suspected: every removed token creates a retrieval/faithfulness problem that must be measured at held task quality.

The sceptical hypothesis in your question is broadly correct: there is no general law saying fewer visible tokens means lower total cost. Savings can move between output, fresh input, cache reads, reasoning, tool recovery, image tokens and rework. The right unit is therefore not “tokens saved”;

it is cost and latency per successfully completed task at held quality. This report uses that as the organising principle.

## Contents

1. 1. How to reason about token-saving claims
2. 2. Ponytail: repository and skill anatomy
3. 3. Ponytail: benchmarks and counter-research
4. 4. Ponytail: installation and best practice
5. 5. Caveman: repository and product anatomy
6. 6. Caveman skill: output compression under scrutiny
7. 7. Caveman Proxy: input compression, recovery and new failure modes
8. 8. Counter-research: what compression research says
9. 9. Direct comparison and decision framework
10. 10. How I would benchmark these tools on a real codebase
11. 11. Practical adoption recommendations
12. Appendix A. Benchmark tables
13. Appendix B. Source register and URLs

# 1. How to reason about token-saving claims

A coding-agent session is not one prompt followed by one answer. It is a repeated loop of system instructions, skill text, repository context, tool schemas, tool results, cached history, model reasoning, assistant prose, code edits and often subagents. A claim such as “65% fewer tokens” is meaningless until the channel and denominator are named.

## 1.1 The token bill has multiple channels

| Channel                | Typical contents                                               | How a “saving” can backfire                                                           |
|------------------------|----------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Fresh input            | new user message, fresh tool result, uncached context          | Compression can delete a decisive detail; model retrieves it later or guesses.        |
| Cache creation / write | context newly placed into provider cache                       | A transform can break cache identity and force new cache writes.                      |
| Cache read             | replayed history, static instructions, tool schemas            | Reducing output may barely matter if cache reads dominate the invoice.                |
| Reasoning / thinking   | hidden or billed reasoning tokens, depending on model/provider | A terse or unusual representation can force more internal reconstruction.             |
| Output prose           | explanations, status updates, summaries                        | Caveman v1 targets this directly; easy to measure, often a minority of agentic spend. |
| Code / patches         | generated files and diffs                                      | Usually cannot be shortened arbitrarily without changing task semantics.              |
| Tool recovery          | follow-up retrieval, MCP calls, re-opened files                | Lossy compression can create extra turns and duplicate context.                       |
| Image/vision tokens    | pixelised text, screenshots                                    | Moving text into pixels changes the billing unit rather than making information free. |

This is why “output tokens fell 65%” and “the whole task cost 65% less” are entirely different claims. Caveman’s current README now says this explicitly: the skill only shrinks output; input and reasoning are untouched; the skill itself adds roughly 1–1.5k input tokens per turn; already-terse workloads can become net-negative. [S09]

## 1.2 The correct objective: savings at held quality

A serious benchmark should optimise a constrained objective rather than token count alone. One useful formulation is:

```
minimise    expected(task_cost)
subject to  success_rate >= baseline - tolerance
            safety_failures <= baseline
            latency and human-review cost reported separately
```

The quality constraint matters because a compressed run that is 40% cheaper but fails 10% more tasks can be economically worse once retries and developer time are counted. Likewise, a shorter patch that passes a narrow unit test but removes required validation is not a saving; it is deferred incident cost.

## 1.3 Evidence hierarchy used in this report

- **Tier 1 — independent paired benchmarks.** Same tasks, treatment/control, task verifiers and enough repetitions to expose variance. JetBrains provides the strongest independent evidence located for both projects. [S06][S13]
- **Tier 2 — maintainers' reproducible benchmarks.** Useful when methods, negative cases, versions and limitations are disclosed. Both repos improved substantially on this dimension. [S03][S11]
- **Tier 3 — real-user reports and issue threads.** Useful for discovering failure modes and adoption friction, but self-selection and lack of controls prevent causal claims. [S14]
- **Tier 4 — stars, forks and install counts.** Evidence of attention and adoption, not correctness or savings. GitHub-star research supports treating stars as a popularity signal rather than a validation metric. [S17]

## 2. Ponytail: repository and skill anatomy

Ponytail describes itself as “the lazy senior developer”: before adding code, the agent asks whether the requirement needs to exist, whether the codebase already has a solution, whether the standard library or platform can do it, whether an installed dependency suffices, and only then writes the minimum new code. The project’s central claim is therefore complexity avoidance, not linguistic compression. [S01][S02]

### 2.1 Popularity and project state

| Metric                        | Observed state                                       | Interpretation                                                                                              |
|-------------------------------|------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Stars                         | ~92k+ in late Jul/early Aug third-party crawls [S07] | Extraordinary attention for a young agent skill; not evidence of effect size.                               |
| Forks                         | 5.6k on GitHub page inspected 12 Aug 2026 [S01]      | Large experimentation surface and likely many downstream variants.                                          |
| Latest tagged release located | v4.8.4, 29 Jun 2026 [S04]                            | Rapid release cadence; installation should be version-pinned in controlled environments.                    |
| Core skill size               | 120 lines / 95 LOC / ~6.5 KB [S02]                   | The behavioural “algorithm” is compact; the surrounding repo handles activation, commands and integrations. |
| Licence                       | MIT for core skill/repo [S02]                        | Low friction for internal adaptation.                                                                       |

#### Popularity caveat

The star count is interesting because it indicates that the problem resonates: developers recognise over-building by coding agents. It should not be read as 92,000 independent validations of the 54% claim. A star can mean “interesting”, “bookmark”, “meme I enjoyed”, “I use this daily”, or several other things. [S17]

## 2.2 The core skill is a decision ladder

14. Does this need to exist at all? If speculative, skip it (YAGNI).
15. Is the capability already present in the codebase? Reuse it.
16. Does the standard library solve it?
17. Does the native platform solve it?
18. Does an already-installed dependency solve it?
19. Can the requirement be one line?
20. Only then, implement the minimum that works.

The important part is the ordering. A date picker is a good demonstration: a generic coding model may treat “date picker” as a component-building task. Ponytail reframes it as a platform-capability question and can stop at `<input type="date">`. The resulting code reduction is not compression; it is a different design decision. [S01][S02]

## 2.3 What prevents “lazy” from becoming careless?

The skill explicitly protects trust-boundary validation, data-loss error handling, security, accessibility and requirements the user actually asked for. It also says that laziness applies to the solution, not to understanding the problem: read the affected code and trace the real flow first. Bug fixes are directed toward the shared root cause rather than a local symptom. [S02]

This distinction is crucial. “Write less code” is dangerous as a standalone instruction because the shortest implementation is often insecure or brittle. Ponytail tries to encode a minimum sufficient implementation, not code golf. Its own control experiment supports the need for that distinction: the bare “YAGNI + one-liners” arm was shorter but failed one safety case. [S03]

## 2.4 Activation architecture matters more than the Markdown

An agent skill only has value if it actually reaches the model. Ponytail ships plugin hooks and agent-specific integration files because relying on semantic self-activation is unreliable. The strongest independent test found that simply installing the skill and allowing Claude Code to decide when to invoke it produced 0 activations in 10 sessions. Their measured results required injection of Ponytail’s own SessionStart ruleset. [S06]

### Technical implication

When benchmarking agent skills, “installed” is not a treatment definition. You must log whether the skill/ruleset was actually present in every treatment turn and absent in baseline. Otherwise a null result may only prove that routing failed.

# 3. Ponytail: benchmarks and counter-research

## 3.1 The original benchmark had a real flaw

Ponytail initially advertised much larger 80–94% code reductions from single-shot completions. Colin Eberhardt’s Scott Logic review showed that the baseline often emitted multiple alternatives and explanatory prose, which inflated its line count. On the same simplistic benchmark, a short “Follow YAGNI principles” instruction nearly matched Ponytail, and adding “one-liner solutions” beat it. [S05]

That critique does not prove Ponytail has no value. It proves the original benchmark was measuring a mixture of implementation minimalism and conversational verbosity. The maintainers accepted the criticism, revised the headline, rebuilt the benchmark around real Claude Code sessions and documented the earlier number as a per-task ceiling rather than an average. [S01][S03]

### 3.2 The rebuilt official agentic benchmark is much better

The 18 June benchmark uses a real open-source FastAPI + React repository, headless Claude Code, Haiku 4.5, twelve feature tickets, four repetitions, a no-skill baseline and explicit controls. It scores the workspace diff rather than counting conversational answer lines. It also includes a separate safety tier. [S03]

| Arm                | LOC  | Tokens | Cost | Time | Safety |
|--------------------|------|--------|------|------|--------|
| Ponytail           | -54% | -22%   | -20% | -27% | 100%   |
| Caveman control    | -20% | +7%    | +3%  | +2%  | 100%   |
| YAGNI + one-liners | -33% | -14%   | -21% | -30% | 95%    |

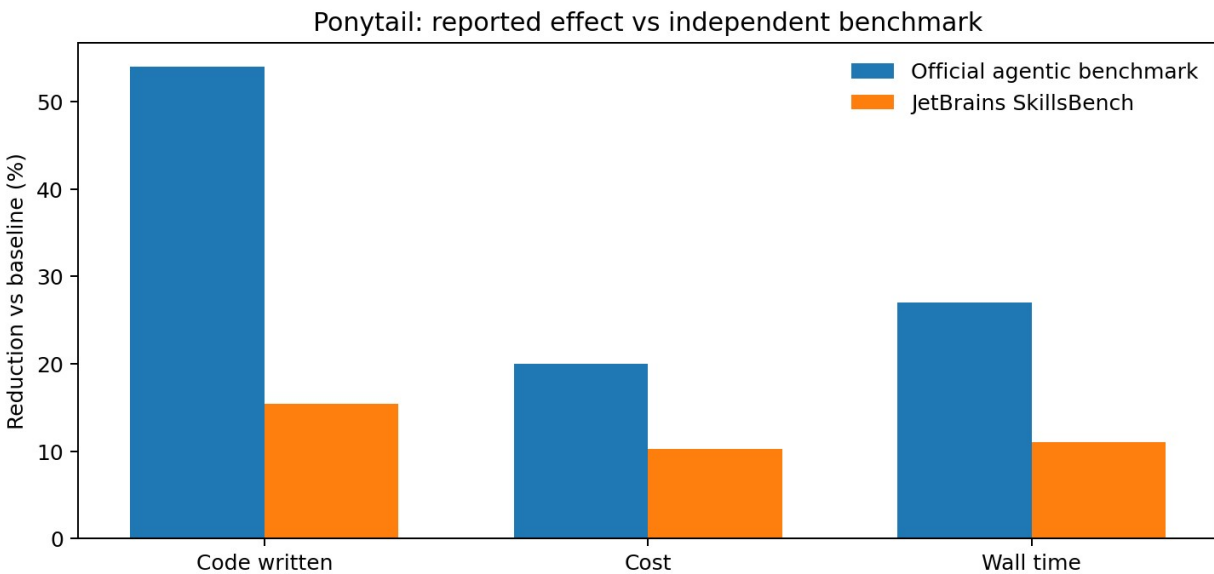
The largest wins are highly interpretable rather than mysterious: date picker 404→23 lines, colour picker 287→23, dropzone 251→95 in the published results. Backend tasks where the implementation was already close to irreducible show little or no reduction. [S03]

The benchmark also discloses a contamination bug in an earlier agentic run: Ponytail's SessionStart hook fired in all arms, including baseline. The maintainers fixed isolation and published the mistake. That candour increases confidence in the corrected result, while also illustrating why prompt/plugin benchmarks are easy to get subtly wrong. [S03]

### 3.3 Independent JetBrains benchmark: smaller, but real

JetBrains ran Ponytail on 80 paired SkillsBench tasks using Claude Code 2.1.201 and Sonnet 5 at medium reasoning. The treatment ruleset was generated from Ponytail's own hook code; every trial was audited for contamination. Across the full run they measured about 15% less code, 10.3% lower cost and 11% lower wall time. The cost signal was statistically significant (reported  $p=0.004$ ). [S06]

Quality did not show a detectable difference: 65 pairs were identical, 9 slightly worse and 6 slightly better. This is a null result rather than a proof of equivalence. JetBrains also explicitly says SkillsBench is not a security/accessibility suite, so it cannot independently validate Ponytail's "100% safe" headline. [S06]



Different task sets/models: comparison is diagnostic, not an apples-to-apples replication.

Figure 1. Ponytail official versus independent effects. Different workloads and models mean the gap should be interpreted as task-distribution sensitivity, not as a failed exact replication. Sources: [S03][S06].

### 3.4 Why the 54% and 15% can both be true

Ponytail has a large treatment effect only when the baseline has room to over-build. Its own benchmark deliberately contains traps where native browser capabilities replace custom components. SkillsBench contains more data, analysis and repair tasks; those often have an irreducible amount of code. JetBrains found reductions reaching roughly 31% on larger builds and almost nothing on already-lean tasks. [S06]

#### Verdict on the Ponytail claim

Do not use "54% less code" as an estate-wide forecast. Use it as evidence that the mechanism can produce very large local wins. A more defensible planning prior from the independent benchmark is roughly 10% lower task cost and 15% less generated/persisted code on a mixed agentic workload, with wide task-level variation.

## 4. Ponytail: installation and best practice

The official repository supports many agents, but the correct installation depends on whether the host supports always-on hooks, skills, rules files or plugins. For Claude Code, the recommended path is the plugin because the SessionStart injection is part of the product behaviour. [S01][S06]

### 4.1 Claude Code

```
/plugin marketplace add DietrichGebert/ponytail
/plugin install ponytail@ponytail
```

The README notes that the plugin lifecycle hooks require Node.js on the non-interactive PATH; if Node is absent the skill files still exist, but always-on activation can silently disappear. [S01]



## 4.2 Other supported paths

- Codex and several other hosts: use the repo's documented plugin/rules route; verify the rules reach the model rather than assuming discovery.
- Gemini/Antigravity, OpenCode, Hermes, Devin, OpenClaw and editor agents have dedicated install instructions in the README/release history. [S01][S04]
- For hosts that only consume AGENTS.md or rule files, copy the smallest scoped ruleset into the project rather than making it global immediately.

## 4.3 Safe adoption pattern

21. Pin a released version or commit; do not benchmark moving `main`.
22. Install at project scope first. Avoid global activation until you know the task types where it helps.
23. Log actual activation in treatment sessions. A skill that did not load is not a treatment.
24. Start with `lite` or `full`, not `ultra`, on unfamiliar repositories.
25. Run repository tests, type checks and security-specific assertions after every task. Do not let "smaller diff" become a quality metric by itself.
26. Measure final diff size and maintainability, not cumulative tool-call text alone.
27. Test tasks where minimalism should not win: validation, permissions, migrations, failure recovery, accessibility, observability and public APIs.

# 5. Caveman: repository and product anatomy

Caveman is now easier to misunderstand because the repository has evolved from one prompt-like skill into an ecosystem. The README explicitly says there are three separate installs: the original shorter-answer skill, Caveman Proxy for smaller inputs, and Caveman Browse. There is also a TypeScript Agent SDK. [S09]

## 5.1 Popularity and project state

| Metric                      | Current observation (12 Aug 2026)                                | Notes                                                                                               |
|-----------------------------|------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Stars                       | 97,629 [S08]                                                     | Exact GitHub REST API snapshot.                                                                     |
| Forks                       | 5,626 [S08]                                                      | Exact API snapshot; GitHub UI rounded to 5.6k.                                                      |
| Open issues + PRs API field | 477 [S08]                                                        | GitHub API `open_issues_count` includes open pull requests.                                         |
| Core skill size             | 88 lines / 63 LOC / 6.07 KB [S10]                                | Small output-style policy; much of current repo complexity lives outside the skill.                 |
| Languages / architecture    | Go-heavy engine/proxy plus JS/TS integration surfaces [S08][S09] | Substantially more than a Markdown skill.                                                           |
| Licence                     | Split MIT + BSL-1.1 [S12]                                        | Skill/SDK/CLI surfaces MIT; compression engine-linked Go code BSL with commercial-hosting boundary. |

## 5.2 Product A: the original Caveman skill

The core skill asks the model to remove articles, filler, pleasantries and hedging; use sentence fragments and short synonyms; avoid tool-call narration; preserve exact technical terms, code, error strings, numbers and negations; and drop the style when compression would create ambiguity or when a security/irreversible warning needs normal prose. [S10]

A particularly good design choice is what it does not do. The current skill forbids invented abbreviations such as ``cfg/impl/req/res/fn`` and even discourages causal arrows when the tokenizer does not save tokens. This directly weakens the concern that it teaches the model a completely invented shorthand. The risk is not “a language the model has never seen”; it is reduced redundancy and less explicit grammar. [S10]

## 5.3 Product B: Caveman Proxy / Engine

The Proxy sits between the existing agent and provider. It classifies payloads and applies content-type-specific transforms. The current README lists JSON, logs, code, diffs, search results and text/HTML with different target reduction ranges. It can also re-encode uniform JSON as TOON when measured smaller, compress shell output, expose retrieval via MCP and fit context to a token budget using relevance/recency/error signals. [S09]

Crucially, this is lossy context transformation with a recovery channel. Before transformed content goes upstream, original bytes are stored in CCR, a local content-addressed recovery store. If the agent needs detail that was elided, it can call ``caveman_retrieve`` / CLI retrieval. Parse/store failures or transformations that are not smaller fall back to the original payload. [S09]

### What “recoverable” really means

Recoverability protects against permanent data destruction; it does not make the first model call semantically lossless. The model must either retain enough information to solve the task or realise that missing information matters and retrieve it. If an omitted detail is both decisive and not recognised as missing, the failure can be silent.

## 5.4 Product C: Pixel and skill-to-PNG modes

Pixel mode renders dense text slabs into PNG pages for vision-capable models. The repo reports an inferred example of 55,413 estimated text tokens becoming 11,402 estimated image tokens on a dense synthetic/request corpus, while explicitly declining sparse code where the image representation is not profitable. Skill conversion similarly preserves skill frontmatter as text but renders the body to pages only when the measured estimate is smaller. [S09]

This is clever engineering, but the economic interpretation changes: information is not disappearing, it is crossing a modality boundary. You must evaluate image-token pricing, visual legibility, OCR-like model errors, layout sensitivity, model support and cache behaviour. The repo itself labels these figures ``inferred``, which is the correct level of certainty. [S09]

## 5.5 Licence boundary is materially different from the original skill

The original ``skills/`` path remains MIT. The engine, proxy, MCP, shrink, browser and related Go core are BSL-1.1. The additional grant permits internal evaluation, development, CI and first-party self-hosted production use, while third-party hosted/managed/embedded optimisation requires a commercial licence. BSL versions convert to Apache 2.0 on the earlier of 21 June 2030 or four years after that version’s first public BSL distribution. [S12]

### **Enterprise implication**

A team can evaluate and self-host the engine for its own traffic under the stated grant, but embedding it into a multi-tenant product or selling the optimisation layer is a different licensing case. Legal review is warranted before productising the Proxy, even though the original skill itself is MIT.

## **6. Caveman skill: output compression under scrutiny**

### **6.1 What the 65% number actually measures**

The official table compares ten prompts against normal verbose replies and reports an average 1,214→294 output tokens, or 65% reduction. Individual tasks range from 22% to 87%. This is a valid measurement of response-length reduction on that prompt set, not an end-to-end coding-agent cost benchmark. [S09]

The current README now contains an unusually important disclaimer: input and reasoning tokens are untouched; the skill adds roughly 1–1.5k input tokens per turn; whole-session savings are smaller; and terse workloads can be net-negative. That caveat should be considered part of the headline, not a footnote. [S09]

### **6.2 Independent JetBrains result: 8.5%, with forced activation**

JetBrains ran 82 paired SkillsBench tasks with Caveman forcibly active. They measured 8.5% fewer output tokens, far below the advertised 65%. The important reason is compositional: coding-agent output contains code, diffs, commands, exact error text and tool interactions that Caveman deliberately preserves. Only the narrative layer is compressible. [S13]

Quality again showed no detectable shift in the benchmark (64 tied, 8 better, 10 worse; reported sign-test  $p=0.82$ ), but the benchmark cannot prove semantic equivalence for every domain. JetBrains also notes that early small samples were much more dramatic and failed to replicate, reinforcing the need for paired tasks at scale. [S13]

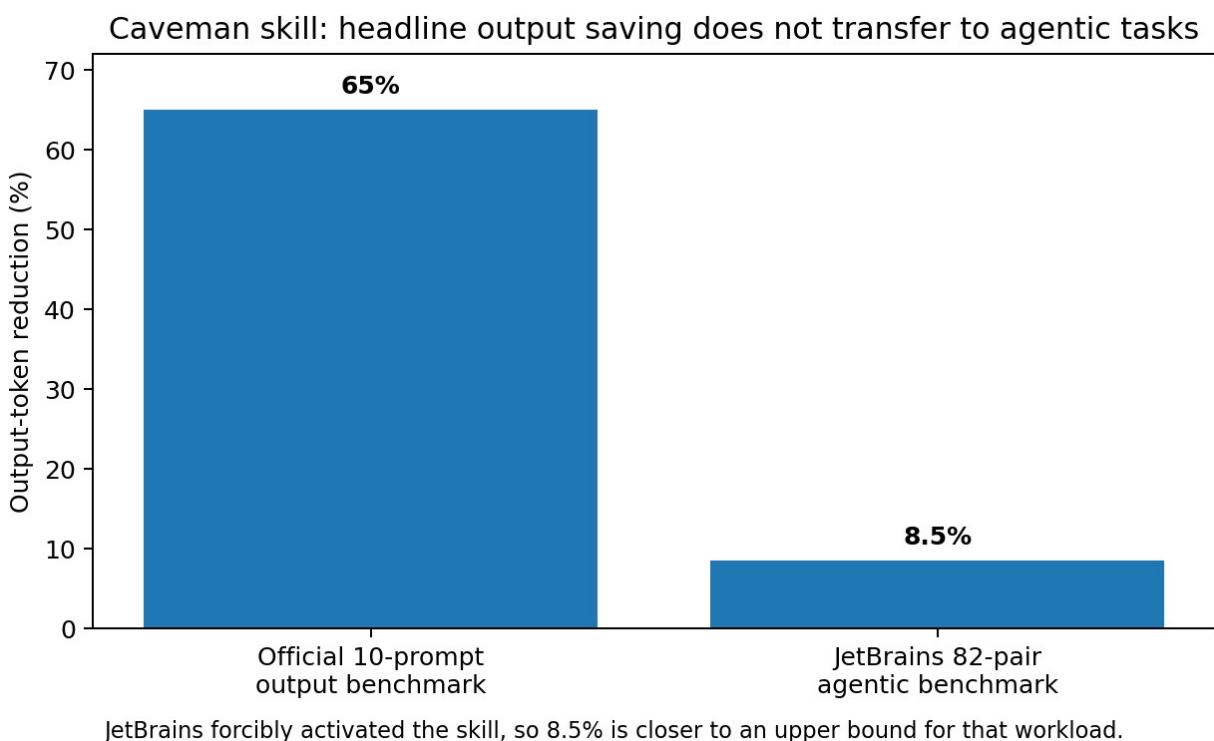


Figure 2. Caveman output-token reduction: the official prose-heavy benchmark versus a broader independent agentic benchmark. Sources: [S09][S13].

### 6.3 “Caveman language” is not the main technical objection

Modern LLM pre-training corpora contain enormous amounts of fragments, headlines, chat, code comments, bullet points, note-taking language and ungrammatical text. It would be too strong to claim that telegraphic English is out-of-distribution in the absolute sense. The current skill also forbids opaque home-made abbreviations and preserves exact technical vocabulary. [S10]

The more defensible concern is semantic redundancy. Function words, conjunctions and explicit sentence structure are not useless overhead; they often disambiguate scope, causality, temporal order, exception and negation. Caveman’s Auto-Clarity rules recognise exactly this and temporarily return to normal prose for security warnings, irreversible actions and ambiguous multi-step sequences. [S10]

### 6.4 Real-user evidence: useful, but weakly causal

A GitHub issue from a Tessel employee reports “non-trivial” token savings, less reading, and a 100% structural/activation review score in their skill-evaluation framework. Their scenario evals were broadly positive with a niche Lite-mode exception. This supports usability and activation quality, but it is not an independent blinded cost benchmark and was posted by an enthusiastic user to the project’s own issue tracker. [S14]

#### Verdict on the 65% claim

The skill demonstrably shortens prose. The 65% number should be read as a channel-specific best-case/benchmark average for prose output, not as expected coding-agent or invoice savings. On the broadest independent agentic evidence located, 8.5% fewer output tokens is the more relevant prior.

## 7. Caveman Proxy: input compression, recovery and new failure modes

### 7.1 Official pinned result: 33.2% provider input reduction

CaveBench Wrap is considerably more rigorous than a local tokenizer screenshot. It uses six immutable 60–95 KB MCP fixtures, three repetitions per arm, Claude Code 2.1.223, Sonnet 5, provider-reported usage counters and an exact semantic JSON oracle. Across 18 direct/Caveman pairs, the wrapped arm used 591,673 input tokens versus 885,793 direct, a 33.2% reduction, with all 18 exact-answer checks passing. The case-clustered 95% interval is 14.6–48.5%. [S11]

The benchmark counts recovery calls and follow-up provider input, includes full Caveman skill prompt overhead and explicitly forbids dropping negative/no-op cases from publication. These are strong methodological choices. [S11]

| Fixture               | Direct input | Caveman input | Reduction | Quality |
|-----------------------|--------------|---------------|-----------|---------|
| SRE log needle        | 148,807      | 74,068        | 50.2%     | 3/3     |
| Deployment JSON drift | 147,975      | 108,939       | 26.4%     | 3/3     |
| Fraud CSV outlier     | 165,823      | 74,484        | 55.1%     | 3/3     |
| Test output failure   | 150,377      | 108,514       | 27.8%     | 3/3     |
| Config YAML drift     | 132,124      | 71,027        | 46.2%     | 3/3     |
| Dashboard HTML alert  | 140,687      | 154,641       | -9.9%     | 3/3     |

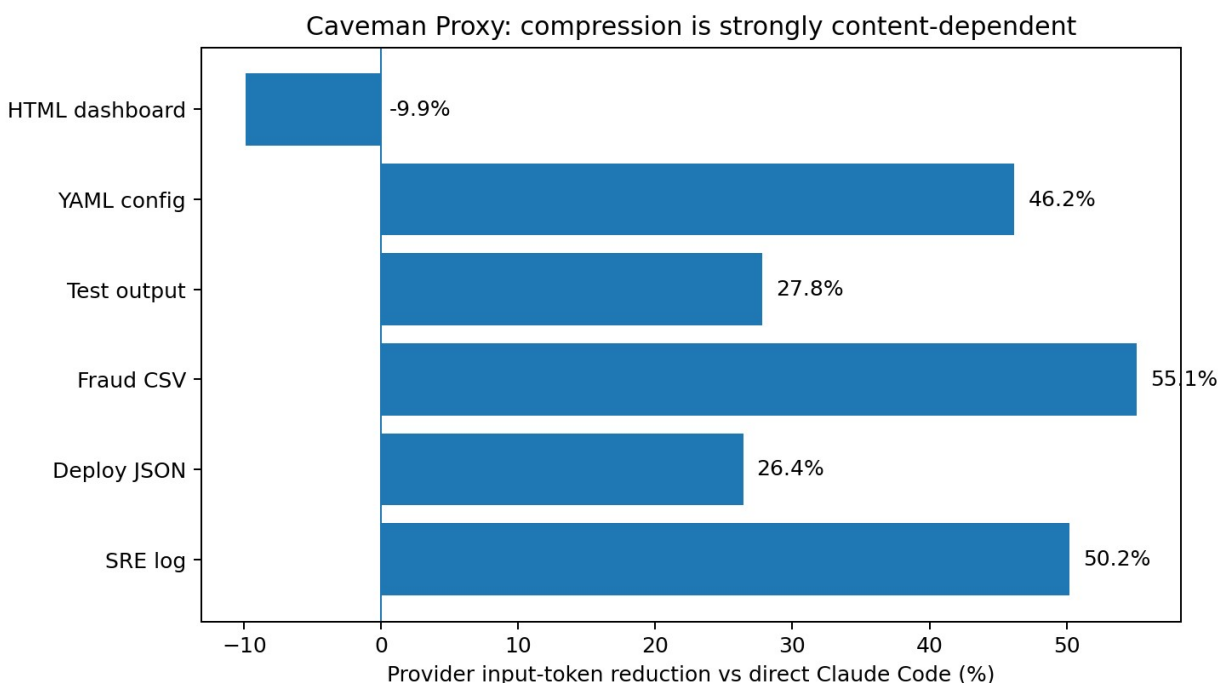


Figure 3. Caveman Proxy's official per-case result. One workload gets worse, highlighting content-dependent economics. Source: [S11].

## 7.2 Why the negative HTML case matters

The HTML fixture had no profitable compression transform, yet still paid the skill/proxy overhead, so provider input increased 9.9%. This is not an embarrassing edge case; it is the core economics of selective compression. Every transformation has fixed metadata, instructions and recovery machinery. If payload redundancy is below that fixed cost, "compression" is expansion. [S11]

The Browse benchmark provides an even sharper example. On a 200-row operations table, a focused query is dramatically smaller than a generic accessibility-tree representation. On a tiny checkout form, Caveman's full representation is larger than Playwright ARIA because action IDs, recovery handles and accounting metadata dominate. The repo publishes this regression. [S09]

## 7.3 Where context loss can appear

| Failure mode                 | Mechanism                                                                                                                           | What to test                                                                                                   |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Dropped low-frequency detail | Log/text compressor retains errors/top/bottom/important spans and removes "noise"; a rare clue can be outside heuristic importance. | Needle-in-haystack tasks where decisive evidence looks ordinary until combined with another fact.              |
| Code-body elision            | Signatures/imports/types retained while bodies can be omitted.                                                                      | Bugs caused by implementation semantics rather than interfaces; side effects; dynamic dispatch; data mutation. |
| Cross-segment reasoning loss | Each payload may be compressed sensibly in isolation but the answer depends on two weak signals across segments.                    | Tasks where evidence is distributed across file + log + config + history.                                      |
| Recovery blindness           | Model cannot request what it does not know exists.                                                                                  | Hide decisive detail and check whether recovery triggers without an explicit hint.                             |
| Extra-turn amplification     | Recovery causes another tool call and model turn.                                                                                   | Measure end-to-end provider tokens and latency, not first request only.                                        |
| Cache disruption             | Transformed representation can alter stable prefixes or cache reuse patterns.                                                       | Report cache creation/read separately; compare warm multi-turn sessions.                                       |
| Modality shift               | Pixel sends glyph images instead of text.                                                                                           | Exact identifiers, punctuation, tiny text, Unicode, tables, code and model-specific vision accuracy.           |
| Proxy operational risk       | Local routing layer, binaries, MCP and credential forwarding add another failure surface.                                           | Outages, version skew, auth passthrough, crash fallback, logging/privacy and upgrade rollback.                 |

## 7.4 "Nothing comes for free": where the cost moves

- **Compute moves local.** Compression, parsing, tree-sitter, BM25 selection, pixel rendering and recovery-store writes consume local CPU/memory even if provider tokens fall.
- **Complexity moves into the middleware.** You now own another versioned layer between agent and provider, with content detection, transforms, MCP tools and fallback rules.

- **Information moves into CCR.** The provider does not see some bytes initially; correctness depends on the visible summary plus retrieval policy.
- **Tokens can move to later turns.** A successful recovery may erase part of the first-call saving, and an unsuccessful silent omission can cost a full retry.
- **Text tokens can become image tokens.** Pixel mode changes modality and price/accuracy behaviour rather than eliminating information.
- **Human time can move.** More compact traces are easier to read when they are sufficient, but harder to debug when the omitted material is precisely what an engineer needs.

## 7.5 Current evidence gap

I did not locate an independent, broad paired benchmark of the current Caveman Proxy comparable to JetBrains' tests of the original skill. The official 33.2% result is well specified but narrow: six deterministic large tool-output tasks, exact answers, one model/harness version, and no open-ended repository-level coding. The authors themselves state these boundaries. [S11]

### Proxy verdict

Technically credible mechanism, promising official evidence, insufficient external evidence for a universal forecast. I would treat 33.2% as a hypothesis for large repetitive tool-output workloads, not as the default expected saving across an engineering organisation.

## 8. Counter-research: what compression research says

### 8.1 CAVEWOMAN directly tests the “caveman” intuition

The June 2026 CAVEWOMAN paper evaluates eight models, five datasets and five reduction levels while separately compressing input and output. The headline result is asymmetric: output compression often reduced realised cost, while linguistic input compression was a “lose-lose” on their benchmark average, increasing net cost to roughly 1.15× while accuracy deteriorated; stronger compression could be much worse. Models often compensated for compressed input with longer responses. [S15]

This is strong evidence for your intuition that a terse representation can shift work into model reconstruction. It should not, however, be misapplied to Caveman Proxy. CAVEWOMAN studies linguistic input rewriting; the Proxy performs structural, content-aware compression with a byte-exact recovery store. They share the information-loss problem but not the same mechanism. [S09] [S15]

### 8.2 SkillReducer supports a different kind of compression

SkillReducer analysed 55,315 public skills and found substantial non-actionable content. Its progressive-disclosure approach compressed skill descriptions by 48% and bodies by 39% while improving functional quality by 2.8% on its evaluation, with transfer across multiple model families. [S16]

This is useful because it shows that “less context” can genuinely be better when what you remove is redundant or distracting. The lesson is not that compression is bad. The lesson is that semantic selection beats indiscriminate terseness. Ponytail's decision ladder and Caveman Proxy's content-aware routing are conceptually stronger than simply deleting grammar everywhere. [S16]

## 8.3 A practical information-theoretic view

Natural language contains redundancy. That redundancy has two functions: it makes communication robust to noise and it resolves ambiguity. Compression is safe while it removes predictable redundancy; it becomes risky when it removes bits that condition the correct decision. The engineering problem is therefore not “how many tokens can I delete?” but “which tokens have low marginal value for this task?”

Ponytail attacks redundancy in the solution space: do not create objects, abstractions or dependencies that the task does not need. Caveman skill attacks redundancy in the communication surface: remove pleasantries and syntactic scaffolding. Caveman Proxy attacks redundancy in the context representation: collapse repetitive structured data and retain/retrieve relevant details. Those are three different compression problems with three different failure modes.

## 9. Direct comparison and decision framework

| Dimension                           | Ponytail                                                               | Caveman skill                                             | Caveman Proxy                                                                        |
|-------------------------------------|------------------------------------------------------------------------|-----------------------------------------------------------|--------------------------------------------------------------------------------------|
| Best use case                       | Agents that routinely over-engineer implementation work                | Humans who want less commentary and slightly lower output | Large tool payloads/logs/JSON/repeated context where provider input dominates        |
| Primary KPI                         | Task success + final diff size/complexity                              | Task success + output tokens + readability                | Task success + provider input/cache tokens + recovery rate                           |
| Independent validation              | Strongest of the three [S06]                                           | Good for original skill [S13]                             | Not yet broad/independent in sources located                                         |
| Quality risk                        | Under-building / missing non-functional requirements                   | Ambiguous terse prose                                     | Context omission / retrieval failure                                                 |
| Operational complexity              | Low-moderate plugin/rules                                              | Low for skill                                             | High: local binaries, proxy, recovery store, MCP, content routing                    |
| Licence complexity                  | Low (MIT)                                                              | Low (MIT skill)                                           | Medium-high (BSL engine boundary) [S12]                                              |
| Expected savings profile            | Sparse: large wins on over-build traps; near zero on irreducible tasks | Bounded by how much assistant prose exists                | Sparse by content type: large on repetitive payloads, negative on small/no-op inputs |
| Would I deploy globally by default? | Only after repo-specific eval                                          | Only if team prefers terse interaction                    | No; first deploy to measured high-input workflows                                    |

### 9.1 Which claim survives scrutiny best?

- **Ponytail — strongest claim:** “This reduces over-building on some coding tasks.” Supported by mechanism, official benchmark and an independent paired benchmark. The exact 54% estate-wide figure is not supported.
- **Caveman skill — strongest claim:** “This makes assistant prose materially shorter.” Strongly supported. “It saves 65% of an agent’s tokens/cost” is not supported by the independent agentic benchmark.



- **Caveman Proxy — strongest claim:** “Selective structural compression can reduce provider input substantially on large repetitive tool outputs while retaining exact answers in a pinned suite.” Supported by official benchmark; generalisation remains open.

## 9.2 Do the tools stack?

They can, but you should not benchmark them stacked first. Ponytail changes what the agent builds, Caveman changes what it says, and the Proxy changes what it reads. If you enable all three at once and cost falls 15%, you will not know which mechanism produced the gain or which one introduced a quality regression. Establish single-treatment effects, then test interactions.

There is also a plausible interaction: Ponytail may make diffs and tool output smaller, reducing the remaining compression headroom for Caveman Proxy. Conversely, a terser Caveman narrative may make Ponytail’s communication-side savings look smaller while code-side savings remain. Additive percentages are therefore not justified.

# 10. How I would benchmark these tools on a real codebase

If the aim is to decide whether either tool belongs in a production data/engineering workflow, I would run a paired, repository-level experiment before changing team defaults. The protocol below is deliberately designed to prevent the most common “token-saver benchmark” mistakes.

## 10.1 Build the task set

Use at least 30–50 tasks for a first serious pass; 80–100 is better if the tasks are cheap enough. Stratify them so the tool cannot win by being tested only where its mechanism is expected to shine.

| Task stratum                | Examples                                               | Why it matters                                                   |
|-----------------------------|--------------------------------------------------------|------------------------------------------------------------------|
| Small fixes                 | null guard, typo, one-file bug                         | Tests irreducible work; Ponytail should approach zero delta.     |
| Over-build traps            | native browser control, stdlib parser, existing helper | Where Ponytail should show the largest treatment effect.         |
| Cross-file features         | API + schema + UI + tests                              | Tests whether minimalism still traces full dependency flow.      |
| Security / validation       | authz, path traversal, input validation, secrets       | Tests the explicit “never cut safety” boundary.                  |
| Data / config / logs        | large JSON, YAML, test failures, SRE logs              | Where Caveman Proxy should have structural compression headroom. |
| Needle tasks                | rare clue in repetitive payload                        | Tests silent context-loss risk.                                  |
| Long multi-turn repair      | agent iterates after test failures                     | Exposes cache, recovery and repeated-context effects.            |
| Documentation / explanation | architecture rationale, incident note                  | Where Caveman output compression is most likely to help.         |

## 10.2 Experimental arms

28. Baseline: agent with no Ponytail/Caveman global rules, clean config.

29. Ponytail only: exact pinned plugin/ruleset; verify activation per run.

30. Caveman skill only: exact pinned skill, forced activation for a ceiling measurement and auto-activation as a separate "realistic" arm.
31. Caveman Proxy only: run the proxy in its default stack but disable output style if you want a pure input-compression estimate.
32. Optional later: Ponytail + Caveman skill; Ponytail + Proxy; full stack.

### 10.3 Pin everything that can move

- Model ID and reasoning effort.
- Agent/harness version.
- Skill/plugin commit or release.
- Repository commit and dependency lockfiles.
- Tool/MCP catalogue.
- Permissions and sandbox policy.
- Environment variables affecting hooks, cache or proxy routing.
- Repetition order: randomise or rotate treatment order to reduce time/provider drift.

### 10.4 Measure the full ledger

| Metric                                           | Why                                                                        |
|--------------------------------------------------|----------------------------------------------------------------------------|
| Task verifier score / pass rate                  | Primary quality constraint.                                                |
| Security-specific verifier results               | General task tests do not validate safety claims.                          |
| Input tokens                                     | Fresh provider input.                                                      |
| Cache creation + cache read separately           | Compression can change cache economics.                                    |
| Reasoning tokens / billed thinking where exposed | Detect reconstruction cost.                                                |
| Output tokens                                    | Direct Caveman-skill target.                                               |
| Total provider cost                              | What finance ultimately sees on API workloads.                             |
| Wall time + tool-call count                      | Recovery and extra turns can erase latency gains.                          |
| Recovery count and recovered bytes               | Proxy-specific signal: how often compression hid necessary detail.         |
| Final diff LOC + changed files                   | Ponytail's direct engineering footprint.                                   |
| Static quality / complexity                      | Cyclomatic complexity, dependencies added, lint/type errors.               |
| Human review time                                | A smaller answer is not helpful if it takes longer to understand or audit. |

### 10.5 Primary estimand

$\text{cost\_per\_success} = \text{total\_provider\_cost} / \text{number\_of\_successful\_tasks}$

$\text{relative\_gain} = 1 - (\text{cost\_per\_success\_treatment} / \text{cost\_per\_success\_baseline})$

Report this with a confidence interval, plus the raw quality delta. If a tool reduces raw tokens but increases failures, cost-per-success exposes the false economy immediately. For subscription agents where token cost is not directly billed, substitute provider-reported token volume and latency while still retaining quality as a hard gate.

## 10.6 Analyse heterogeneity, not only the mean

Both projects have strongly heterogeneous effects. Aggregate averages can hide this. Report per-task deltas and group them by task stratum and payload type. A tool that saves 45% on logs, loses 10% on HTML and does nothing on small code can still be excellent if routing knows when to activate it. Conversely, a 12% overall average can hide catastrophic regressions in exactly the tasks you care about.

### The benchmark I would trust

A paired task set from your own repositories, 3–5 repetitions, verified treatment activation, provider-reported token buckets, tests/security gates, recovery accounting and cost-per-success with bootstrap confidence intervals. Anything materially simpler is a demo, not due diligence.

## 11. Practical adoption recommendations

### 11.1 If you want to try Ponytail

- 33. Install the pinned plugin in one project and ensure Node/hook activation works.
- 34. Use `full` first. Keep `ultra` for experiments rather than team default.
- 35. Create a small eval set containing both over-build traps and safety-critical tasks.
- 36. Look at the final diff: dependencies, files, abstractions and maintenance burden matter more than token count.
- 37. If your baseline agent is already disciplined and minimal, expect little gain. That is not a failure; it means the problem Ponytail addresses is absent.

### 11.2 If you want to try the Caveman skill

- 38. Install only the MIT skill first, preferably pinned to a release (the current README demonstrates v1.10.0). [S09]
- 39. Use `lite` or `full` if human readability matters; treat `ultra` and Wenyan as specialised modes.
- 40. Measure output-token savings separately from total task cost.
- 41. Do not compress persisted docs, commit messages or external communication unless you explicitly want that style; the core skill itself keeps persisted artefacts in normal prose. [S10]
- 42. Expect the largest gain in explanation-heavy sessions, not code-heavy agent loops.

### 11.3 If you want to try Caveman Proxy

- 43. Start in metering/pass-through mode (`wrap --off`) to learn where your context bill actually comes from before enabling transforms. [S09]
- 44. Enable it first for large logs, JSON, test output or repetitive tool payloads, not every workflow.
- 45. Keep CCR local, inspect storage/retention permissions and understand how credentials are passed through the local proxy.
- 46. Add explicit recovery metrics and alerts. A high recovery rate may mean the compressor is too aggressive; a suspiciously zero recovery rate may mean the model is failing silently rather than that compression is perfect.
- 47. Run needle tests where omitted details are decisive.
- 48. Verify cache economics in long sessions. Input token reduction alone is not enough if transformed prefixes reduce cache reuse.

49. Get legal review if the engine is embedded into a product or third-party service because the BSL boundary differs from the MIT skill. [S12]

## 11.4 Final ranking

| Use case                                     | Recommendation         | Reason                                                                                                                      |
|----------------------------------------------|------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Reduce agent over-engineering                | Try Ponytail first     | Independent evidence supports modest cost/code gains; mechanism improves maintainability even when token savings are small. |
| Reduce noisy commentary                      | Try Caveman skill      | Low operational cost; very likely to make prose shorter; do not budget around 65%.                                          |
| Reduce very large tool-context bills         | Evaluate Caveman Proxy | Most technically targeted mechanism; official result promising, but requires workload-specific validation.                  |
| Organisation-wide default without local eval | Do not do this yet     | All observed effects are task-distribution dependent; activation, model version and payload shape matter.                   |
| Stack everything immediately                 | Avoid                  | Confounds attribution and can hide interaction failures.                                                                    |

### Final conclusion

The interesting story is not that these projects are fake. Both encode useful ideas. The more important story is that their headline percentages live at different layers of the agent stack. Ponytail's strongest case is software simplicity, Caveman v1's is interface brevity, and Caveman Proxy's is structural context reduction. Once those layers are separated, the evidence becomes much easier to interpret — and much less magical.

# Appendix A. Benchmark tables

## A1. Ponytail official vs independent summary

| Metric     | Official Ponytail agentic benchmark [S03]               | Independent JetBrains [S06]                                                            | Interpretation                                                     |
|------------|---------------------------------------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| Code / LOC | -54% mean over 12 feature tasks                         | ~ -15% typical overall; ~ -31% on larger builds                                        | Large sensitivity to task set and over-build headroom.             |
| Tokens     | -22%                                                    | Fresh/re-read token components smaller but effect less clean; cost moved significantly | Token buckets differ; do not over-compare one aggregate.           |
| Cost       | -20%                                                    | -10.3%, reported p=0.004                                                               | Independent evidence confirms a smaller but real cost effect.      |
| Time       | -27%                                                    | -11%                                                                                   | Directionally consistent.                                          |
| Quality    | 100% safety tier                                        | 65 tied / 9 worse / 6 better verifier scores                                           | Independent suite not designed to validate security/accessibility. |
| Model      | Haiku 4.5                                               | Sonnet 5, medium reasoning                                                             | Stronger/different baseline changes headroom.                      |
| Task set   | FastAPI + React features with explicit over-build traps | SkillsBench data/analysis/repair mix                                                   | Explains much of effect-size gap.                                  |

## A2. Caveman original skill evidence

| Evidence                        | Tasks                              | Activation         | Measured saving                                   | Quality caveat                                                                 |
|---------------------------------|------------------------------------|--------------------|---------------------------------------------------|--------------------------------------------------------------------------------|
| Official output benchmark [S09] | 10 prose/coding prompts            | Skill active       | 65% average output-token reduction (22–87% range) | Not end-to-end agent cost; input/reasoning unchanged.                          |
| JetBrains SkillsBench [S13]     | 82 paired agentic tasks            | Forcibly active    | 8.5% output-token reduction                       | No detectable quality shift; forced activation is a ceiling for that workload. |
| Tessl user report [S14]         | Scenario evals + structural review | Skill used in team | "Non-trivial" savings, no comparable %            | Positive user report, not controlled cost study.                               |

## A3. Caveman Proxy pinned official benchmark

| Statistic                            | Value          |
|--------------------------------------|----------------|
| Direct provider input on paired runs | 885,793 tokens |
| Caveman provider input               | 591,673 tokens |
| Aggregate reduction                  | 33.2%          |

| Statistic                   | Value                                                            |
|-----------------------------|------------------------------------------------------------------|
| 95% case-clustered interval | 14.6% to 48.5%                                                   |
| Exact-answer quality        | 18/18 direct/Caveman held pairs passed                           |
| Runs                        | 54 total across direct, Caveman and Headroom arms                |
| Fixtures                    | 6 deterministic MCP outputs, each 60–95 KB                       |
| Repetitions                 | 3 per case per arm                                               |
| Model / agent               | claude-sonnet-5 / Claude Code 2.1.223                            |
| Negative case               | HTML: -9.9% reduction (i.e. input increased)                     |
| Claim boundary              | Controlled benchmark; not production traffic or universal saving |

## Appendix B. Source register and URLs

All web sources below were inspected for this report; accessed 12 August 2026 unless otherwise stated. "Official" means controlled by the project/maintainer or GitHub platform, not independently validated. URLs are included so every claim can be traced back to its origin.

### [S01] Ponytail repository README (current main)

Official. Primary description, install paths, headline and corrected benchmark claims.

<https://github.com/DietrichGebert/ponytail/blob/main/README.md>

### [S02] Ponytail core SKILL.md

Official. Exact behavioural rules, scope, safety boundaries and intensity levels.

<https://github.com/DietrichGebert/ponytail/blob/main/skills/ponytail/SKILL.md>

### [S03] Ponytail agentic benchmark, 18 June 2026

Official benchmark. 12 feature tasks plus safety tier; n=4; Haiku 4.5; documents benchmark redesign and contamination bug.

<https://github.com/DietrichGebert/ponytail/blob/main/benchmarks/results/2026-06-18-agentic.md>

### [S04] Ponytail releases

Official. Release history; v4.8.4 and plugin/agent support.

<https://github.com/DietrichGebert/ponytail/releases>

### [S05] Colin Eberhardt, "Ponytail? YAGNI!"

Independent critique. Critique of original single-shot benchmark; simple YAGNI prompt matched/beat original score; author later revised benchmark.

<https://blog.scottlogic.com/2026/06/16/ponytail-yagni-and-the-problem-with-prompt-benchmarks.html>

### [S06] JetBrains, "Ponytail Skill for Claude Code: Does It Really Cut Agent Code by 54%?"

Independent benchmark. 80 paired SkillsBench tasks; Sonnet 5; measures code, cost, time and verifier quality.

<https://blog.jetbrains.com/ai/2026/07/ponytail-skill-claude-tested/>

### [S07] Star History: DietrichGebert/ponytail

Third-party telemetry. Approximately 92.1k stars in last-week crawl; GitHub current page showed 5.6k forks.

<https://www.star-history.com/dietrichgebert/ponytail/>

### [S08] Caveman GitHub REST repository metadata

Official platform metadata. Current 12 Aug 2026 snapshot: 97,629 stars, 5,626 forks, 477 open issues+PRs API count.

<https://api.github.com/repos/JuliusBrussee/caveman>

### [S09] Caveman repository README (current main)

Official. Current architecture separates skill, Proxy/Engine, Browse, Pixel and Agent SDK; installation and claims.

<https://github.com/JuliusBrussee/caveman/blob/main/README.md>

### [S10] Caveman core SKILL.md

Official. Exact output-compression rules, safety/clarity gates, intensity levels, language handling.

<https://github.com/JuliusBrussee/caveman/blob/main/skills/caveman/SKILL.md>

### [S11] Caveman CaveBench Wrap benchmark

Official benchmark. 54 runs; six deterministic 60–95 KB tool-output fixtures; 33.2% provider-input reduction; exact-answer gates.

<https://github.com/JuliusBrussee/caveman/blob/main/docs/WRAP-BENCHMARK.md>

### [S12] Caveman licensing map

Official. Split MIT/BSL-1.1 licensing and commercial boundary for engine-linked components.

<https://github.com/JuliusBrussee/caveman/blob/main/LICENSING.md>

### [S13] JetBrains, "Does Speaking to Agents Like Cavemen Really Save 65% of Tokens?"

Independent benchmark. 82 paired SkillsBench tasks; forced activation; measured 8.5% output-token reduction, no detectable quality shift.

<https://blog.jetbrains.com/ai/2026/07/speak-to-ai-agents-like-cavemen-tosave-tokens/>

**[S14] Caveman GitHub issue #109: external skill eval/user report**

User / third-party evaluation. Tessel employee reports non-trivial savings and 100% structural/activation review score; useful adoption evidence, not neutral cost benchmark.

<https://github.com/JuliusBrussee/caveman/issues/109>

**[S15] CAVEWOMAN: How LLMs Behave Under Linguistic Input and Output Compression**

Academic. Eight models, five datasets, five reduction levels; output compression often saves, linguistic input compression can raise cost and reduce accuracy.

<https://arxiv.org/abs/2606.24083>

**[S16] SkillReducer: Optimizing LLM Agent Skills for Token Efficiency**

Academic. 55,315-skill corpus; progressive disclosure and removal of non-actionable skill content reduced tokens while preserving/improving task quality.

<https://arxiv.org/abs/2603.29919>

**[S17] What's in a GitHub Star?**

Academic. Stars are a popularity/interest signal and influence adoption; not a validation metric for technical claims.

<https://arxiv.org/abs/1811.07643>

## B1. Source-quality notes

- GitHub READMEs and benchmark files are the source of truth for what maintainers currently claim, but not independent proof.
- GitHub API was used for Caveman's exact current stars/forks. Ponytail's current UI exposed 5.6k forks but not an exact star number to the crawler; the report therefore uses an approximate ~92k+ figure from recent third-party crawls rather than inventing false precision.
- JetBrains benchmarks are independent of the repositories and use paired task execution with verifiers; they are the highest-weight external evidence in this report.
- Scott Logic critiques the earlier Ponytail benchmark. It should not be used to dismiss the corrected 18 June benchmark; its value is historical and methodological.
- CAVEWOMAN studies linguistic compression and should not be treated as a direct benchmark of Caveman Proxy's structural/recoverable compression.
- GitHub issues and marketplace adoption figures are qualitative evidence only; they are intentionally not averaged into performance claims.

## B2. Interpretation conventions

Percent reductions are presented with the denominator used by the source. "Input", "output", "cost", "time" and "LOC" are not interchangeable. Where two benchmarks use different models, tasks or accounting, charts are labelled as diagnostic comparisons rather than exact replications. Claims described as "inferred" remain estimates; claims described as provider-reported use provider usage counters; neither is automatically equivalent to a paid invoice.